

P1782R1: Local contract restrictions

Audience: SG21

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

July 31, 2019

History

r1:

- Made static conditions unevaluated operands
- Replaced halt with continue; removed the global continuation mode
- Explained behavior subset structure
- Listed meaningful modifier combinations
- Extended comparisons to other papers; removed erroneous claim about P1290R1's continue

Introduction

Disagreement over the mechanisms for controlling contract evaluation and assumption led to removing the feature altogether for C++20; see my [P1494R0](#) as well as [P1490R0](#) for a review of some recent concerns involving optimization. Papers like [P1332R0](#) (pared down as [P1429R2](#) and a bit further as [P1607R0](#)) have also proposed significant extensions to those mechanisms for the purposes of better control and scalability.

Fundamentally, a contract (that is, what N4820 called a *contract-attribute-specifier*) is a boolean expression whose evaluation and significance can be controlled non-locally (by, *e.g.*, the continuation mode). The common protocol (across many files of diverse authorship) for such control is the principal reason that that contracts exist as a language feature. The alternative, per-library approach is of course already available:

```
namespace my_lib {
    struct Stuff { /* ... */ };

    bool check_expensive(), keep_going();
    bool bad_news(const Stuff&);

    void do_stuff(const Stuff &s) {
        if(check_expensive() && bad_news(s))
            if(keep_going()) std::fprintf(/* ... */);
            else throw /* ... */;
        s.do_it();
    }
}
```

It is also worth noting that concerns about optimization and assumptions in particular are subject to the normal implementation-supplied control over such analyses (`-fno-assume-contracts` wouldn't surprise anyone and would be implied by `-O0`). All that being said, the usability concerns that led to the feature's deferral are very real.

The situation can be understood in terms of the four behaviors described in P1429R2. In any given build, a single contract must have exactly one of the four. For some contracts, all four behaviors are reasonable, and the choice is made by the global controls. For others, certain behaviors are unreasonable and should not be selected

regardless of the general preference expressed via the global controls. Perhaps the best example is a contract just introduced into code already in use, where using it to drive optimization would be inappropriate even if other similar contracts are already being so utilized. Another is a contract introduced to prevent following undefined behavior (when checking can be afforded), where continuing past a violation would be nonsensical even if other contracts are being used for logging purposes. Unfortunately, the single local control on a contract (the level) is able to influence the choice of behavior in only a very limited fashion and does not address either of these examples.

This paper proposes replacing the N4820 *contract-level* with a set of *restrictions* that limit the effect of the global controls, to be applied when local conditions (including the recency of a contract's introduction) make it unsafe to take full advantage of the consistent control. The local restrictions only reduce the set of behaviors available via the global controls for a construct; the only direct control (along the lines of [P1334R0](#)) provided is a guaranteed check. This proposal also stops well short of an extensible roles system as proposed by P1332R0.

Two simple changes are proposed to the global controls as well: the global continuation mode is replaced with a local control, and the [ever popular assumption mode](#) is added.

Proposal

Relative to N4820.

Global

Remove the violation continuation mode; except as noted below, call `std::terminate` if the violation handler returns. (The situation with the continuation mode *off* can of course typically be emulated by calling `std::terminate` from the violation handler.) Add a different global control, the *assumption mode*, that when *off* restricts contract condition evaluation to those that are guaranteed to be evaluated. That is, it suppresses the controversial passage from `[dcl.attr.contract.check]/4`:

- it is unspecified whether the predicate for a contract that is not checked under the current build level is evaluated; if the predicate of such a contract would evaluate to `false`, the behavior is undefined.

Local

In the place where one of `default`, `audit`, or `axiom` may currently appear, instead allow a combination of the following modifiers with the given semantics:

<code>tentative</code>	the expression may not be assumed if it is not evaluated; disallows <i>assume</i> behavior
<code>continue</code>	do not call <code>std::terminate</code> after calling the violation handler; replaces <i>enforce</i> behavior with <i>inform</i>
<code>static</code>	the expression is never evaluated and is an unevaluated operand; disallows <i>inform</i> and <i>enforce</i> behaviors
<code>audit</code>	the expression may not be evaluated if the build level is not <i>audit</i> ; conditionally disallows <i>inform</i> and <i>enforce</i> behaviors
<code>always</code>	the expression is always evaluated at all build levels; disallows <i>ignore</i> and <i>assume</i> behaviors

A new contract may be introduced with the `tentative` restriction to avoid adding undefined behavior to existing interfaces. A contract without `continue` may be used to optimize the code that follows it if it is checked, regardless of `tentative` and the global *assumption mode*. The `static` restriction replaces the use of `axiom` for static analysis (and suppresses odr-use), removing the overloading with its use for assumptions for optimization. The role played by `audit` is much the same as in the current draft. The principal purpose of `always` is to allow

the contract syntax to be used with no global control at all (in which case the only reasonable behavior is checking).

The meaningful combinations of the modifiers and the sets of behaviors they supply are

1. (none): {*ignore*, *assume*, *enforce*}
2. tentative: {*ignore*, *enforce*}
3. continue: {*ignore*, *assume*, *inform*}
4. tentative continue: {*ignore*, *inform*}
5. static: {*ignore*, *assume*}
6. static tentative: {*ignore*}
7. audit: {*ignore*, *assume*, *enforce*?}
8. audit tentative: {*ignore*, *enforce*?}
9. audit continue: {*ignore*, *assume*, *inform*?}
10. audit tentative continue: {*ignore*, *inform*?}
11. always: {*enforce*}
12. always continue: {*inform*}

Because the build level makes audit equivalent either to static or to nothing, there are 8 unique sets available. It would be reasonable to make certain combinations like `always static` ill-formed, but note that `static audit` is just equivalent to `static` and merits at most a warning.

Use cases

The safety goals of [P1290R3](#) can be achieved with the global assumption mode.

P1332R0 proposes several features to satisfy its extensive set of use cases. Several of these (that also appear in P1429R2) are covered by this simpler proposal:

1. The `ignore` semantic as a choice for a contract level is provided via the global assumption mode.
2. The use of the `%review` tag to suppress uncontrolled optimizations and prevent crashes from new contracts is addressed by the `tentative` restriction and `continue` modifier respectively.
3. The manual choices of semantics (which of course provide access to certain singleton sets of behaviors) are supported:
 1. `ignore: tentative static` (though less likely to be ignored by a static analyzer), or just `static` with the global assumption mode *off*
 2. `assume: static`
 3. `check_maybe_continue: always continue` (but see below about optimization)
 4. `check_never_continue: always`
4. The mixed treatment of `audit` and default contracts is supported by adding `continue` to checks past which it is known to be safe to continue.

Most of the others may be addressed elsewhere:

1. The modes per-library may be much easier to realize as per-module (since per-translation-unit doesn't mix well with `#include`).
2. The `check_maybe_continue` optimization barrier is realized more generally by `std::observable` from P1494R0.

`check_always_continue` is not addressed, but it appears to be outside the scope of the language to restrict optimization in such a fashion.

Acknowledgments

Thanks to John Lakos for a detailed review of the first published version. Thanks to Joshua Berne for writing P1807R0, which discussed this paper and inspired improvements.